Chapter 13 worked solutions • I/O systems

OS13-E01 • PIO publication and lost wakeups

1. CPU sets READY; controller observes it, sets BUSY, and reads old DATA; CPU only then writes new DATA. Completion cannot repair the command's old input. Required I/O accessors/barriers must make data visible before command publication.
2. Waiter reads false; IRQ sets true and wakes before any waiter exists; waiter sleeps. Under the same lock protect predicate and wait queue. While !done, enqueue and atomically release the lock while sleeping. IRQ sets done/wakes under corresponding locking. The while loop handles unrelated wakeups. IRQ count is not a completion predicate.

OS13-E02 • Partial vector writes

1. First consume120 then50, leaving[base1+50,30],[base2,300], total330. Next consume30 then100, leaving[base2+100,200]. Delivered300 plus remaining200 equals original500.
2. Retain that progress and wait for writability or follow cancellation rules. Resending duplicates300 B. EAGAIN does not undo earlier successful transfers.

OS13-E03 • Virtual timers

1. Initially program7. Insertion gives[6D,7A,10 B,10C], so reprogram6; canceling A leaves6 unchanged. At6 remove D and program10; at 10 remove B and C. A late handler processes every deadline≤now, not only one equal to now.
2. No. Hardware/IRQ delay, deferred work, and scheduling can delay callback execution. The trigger request is not unconditional user-thread execution at an exact instant.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 13 worked solutions • I/O systems (continued)

OS13-E04 • DMA cancellation and reclamation

1. Hardware may still write dma_addr. Reallocated pages could belong to another process. Only confirmed cessation permits synchronization, unmapping, and release.
2. Under common lock/CAS, change IN_FLIGHT to DONE or ABORTED. Only the successful transition publishes/unmaps. Two winners would require the second to succeed from an already-terminal state, contradicting its precondition. This safety proof does not guarantee a broken device eventually responds.
3. No. Late abort/timer/IRQ callbacks may still inspect state and must already own stable references. With owner/IRQ/abort count3, IRQ exit gives2, owner exit1, abort observing DONE and exiting0; only then free. User timeout return, DMA release, and control-block reclamation are different events.

OS13-E05 • Finite double-buffer batch

1. A 1 inputs0–3/processes3–8; B2 inputs3–6/processes8–13; A 3 inputs8–11/processes13–18; B4 inputs13–16/processes18–23. Input waits for a free slot.
2. One buffer takes 4(3+5)=32 ms. Two take3+5+3 • 5=23 ms. Steady period5 ms is CPU-limited. Four times5=20 omits initial 3 ms filling. Input and processing cannot overlap within one buffer, only across buffers.

OS13-E06 • STREAMS watermarks

1. Acceptance reaches900 and stops upstream. One service yields600, still not below400; another yields300 and back-enables. This model explicitly uses strictly below, rather than another implementation's≤.
2. Downstream300 plus upstream600 uses at least900 B. Backpressure distributes waiting. An unpausable source still needs dropping/throttling if all queues fill. Head blocking does not authorize indefinite IRQ waiting.

Chapter 13 worked solutions • I/O systems (continued)

OS13-E07 • Resource bounds and Little's relation

1. Device60000/s; CPU=1/(10 • 10^{-6})=100000/s; memory=2 • 2^{30}/(24 • 2^{10})=87381.33/s. Minimum60000/s is the upper bound, limited by device. Bounds need not be attained.
2. L=50000 • 0.0004=20; in service=50000 • 0.0001=5; queue15; wait0.0003 s. All refer to one steady workload and boundary.
3. No. At least10000 requests/s accumulate, invalidating finite steady state. Fixed L=20 cannot follow from Little's relation when the queue grows without bound. GiB means2^{30} B.

OS13-E08 • Naming, LBA, and durability

1. 12345=3 • 4096+57. PartitionLBA=90 • 8=720; whole-disk start1744. Requested bytes57–156 remain inside the block. A whole-block cache fill reads sectors1744–1751; a 100 B request does not imply100 B hardware I/O.
2. No. The API accepted100 B, possibly only in dirty RAM or volatile controller cache. Compatible filesystem synchronization and device flush/FUA contracts establish durability.
3. No. Without persistent queryable device deduplication/progress, the same host ID can print again. Missing acknowledgment cannot distinguish unprinted from printed. Add a device contract or report outcome unknown. Naming, caching, device completion, and durability have distinct boundaries.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 13 worked solutions • I/O systems (continued)

Verification code appendix: io_models.py

```
"""Finite examples, not proofs of real driver correctness."""
from itertools import permutations

def consume(lengths, n):
    if n<0 or n>sum(lengths):raise ValueError('invalid result length')
    for i,length in enumerate(lengths):
        if n<length:return i,n,[length-n]+lengths[i+1:]
        n-=length
    return len(lengths),0,[]

def double_buffer(n, tin, tcpu):
    slots=[0,0];device=cpu=0;trace=[]
    for i in range(n):
        slot=i%2;begin=max(device,slots[slot]);end=begin+tin;device=end
        p=max(cpu,end);cpu=p+tcpu;slots[slot]=cpu;trace.append((begin,end,p,cpu))
    return trace

def terminal(events):
    state='IN_FLIGHT';publish=0;freed=False
    for event in events:
        if event=='timeout':continue # not evidence of quiescence
        if state=='IN_FLIGHT':
            state='DONE' if event=='completion' else 'ABORTED';publish+=1;freed=True
    return state,publish,freed

def control_lifetime():
    refs={'owner','IRQ','abort'}
    trace=[len(refs)]
    for actor in ['IRQ','owner','abort']:
        assert actor in refs # callback owns a reference before accessing state
        refs.remove(actor);trace.append(len(refs))
    return trace
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 13 worked solutions • I/O systems (continued)

```
class PIO:
    def __init__(self):self.busy=0;self.ready=0;self.data=None;self.delivered=[]
    def submit(self,value):
        if self.busy or self.ready:return False
        self.data=value;self.ready=1;return True
    def accept(self):
        assert self.ready and not self.busy
        self.busy=1;self.delivered.append(self.data)
    def complete(self):
        assert self.busy
        self.ready=self.busy=0

if __name__=='__main__':
    p=PIO();assert p.submit(0x41)
    assert p.busy==0 and p.ready==1
    assert not p.submit(0x42) and p.data==0x41
    p.accept();assert not p.submit(0x42)
    p.complete();assert p.submit(0x42)
    p.accept();p.complete();assert p.delivered==[0x41,0x42]
    print('PIO two submissions: READY=1/BUSY=0 prevents overwrite; delivered [0x41,0x42]')
    assert consume([120,80,300],170)==(1,50,[30,300])
    assert consume([30,300],130)==(1,100,[200])
    t=double_buffer(3,4,6);assert t==[(0,4,4,10),(4,8,10,16),(10,14,16,22)]
    assert double_buffer(4,3,5)[-1][-1]==23
    assert terminal(['timeout'])==('IN_FLIGHT',0,False)
    assert control_lifetime()==[3,2,1,0]
    for order in permutations(['timeout','completion','abort_ack']):
        assert terminal(order)[1:]==(1,True)
    print('iovec remainders:',consume([120,80,300],170),consume([30,300],130))
    print('double-buffer trace:',t)
    print('6 terminal-event orders: at most one publication; timeout alone keeps DMA alive')
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 13 worked solutions • I/O systems (continued)

Verification code appendix: io_timeline.py

```
"""Compose queue, device, and completion latency.

Run:
    python io_timeline.py
"""

from dataclasses import dataclass


@dataclass(frozen=True)
class Request:
    queued_at: int
    device_ns: int


def simulate(requests: list[Request]) -> list[tuple[int, int]]:
    now = 0
    result: list[tuple[int, int]] = []
    for request in requests:
        start = max(now, request.queued_at)
        finish = start + request.device_ns
        result.append((start, finish))
        now = finish
    return result


def main() -> None:
    timeline = simulate([Request(0, 5), Request(2, 3), Request(20, 4)])
    assert timeline == [(0, 5), (5, 8), (20, 24)]
    total_wait = sum(start - req.queued_at for (start, _), req in zip(timeline, [Request(0,
5), Request(2, 3), Request(20, 4)]))
    assert total_wait == 3
    print("timeline", timeline, "queue_wait", total_wait)


if __name__ == "__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study